ITU-MiniTwit - Group e - Report

DevOps, Software Evolution and Software Maintenance, BSc (Spring 2026) **Course code:** BSDSESM1KU

Charlotte Planteig cpla@itu.dk
Frederik Hørup Petersen frap@itu.dk
Marie Johansen majoh@itu.dk
Nikolej Lundquist nivl@itu.dk
Sara Bagger salb@itu.dk Vitus Brodersen vitb@itu.dk

May 21, 2026

Contents

1 System

1.1 Architecture and Design

Authors: Nikolej

This project is a forked repository of the Chirp! project (Planteig et al. 2025) developed for the course "Analysis, Design and Software Architecture (Autumn 2025)". The MiniTwit application architecture, namely

the domain model and codebase structure, is inherited unchanged from the Chirp! project. Hence, we refer to the diagrams compiled in the Chirp! project report, inserted below for convenience.

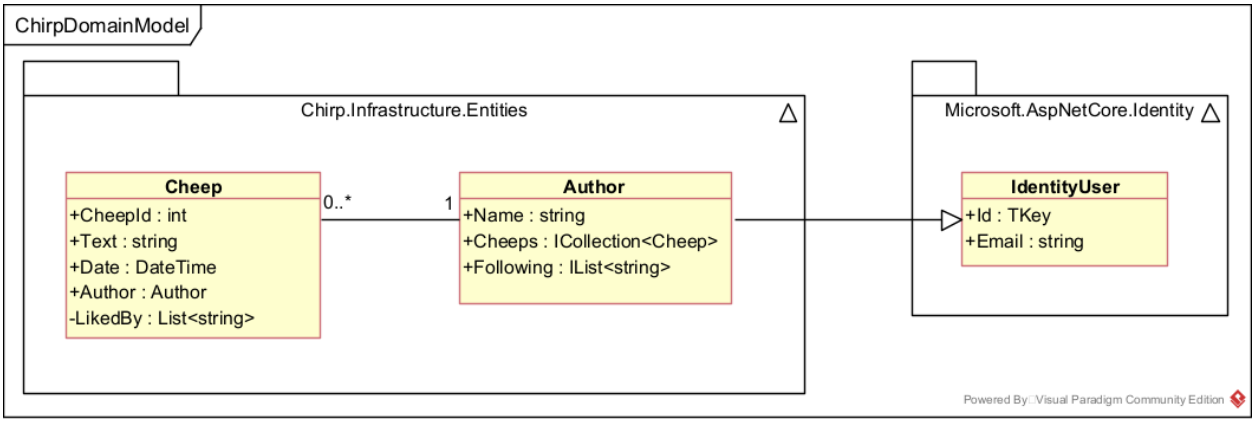


Illustration of the Chirp! *Domain Model (reused from the Chirp! project - not compiled during this course).*

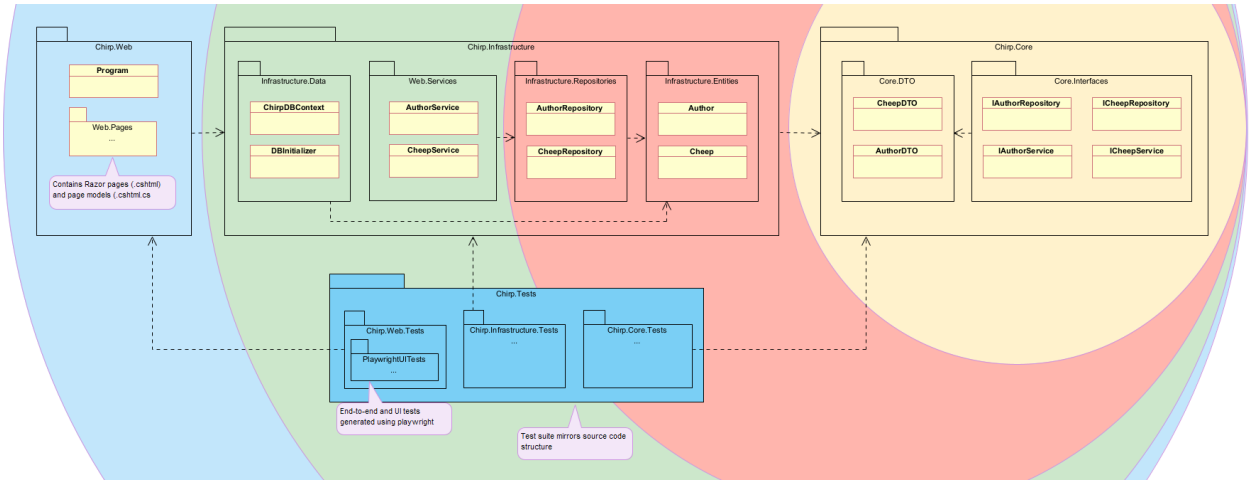


Illustration of the Chirp! app codebase structure - based on onion architecture (reused from the Chirp! project - not compiled during this course).

1.2 Deployment

Authors: Nikolej

Below is a diagram of the overall deployment architecture of the MiniTwit application. The system uses two load balancers and two production server instances each running all containerized services and both read and write to the same DigitalOcean Managed Database.

This setup is designed to prioritize availability. One can imagine the stakeholders of a social media platform pushing for availability, since every second of downtime is potential earnings lost.

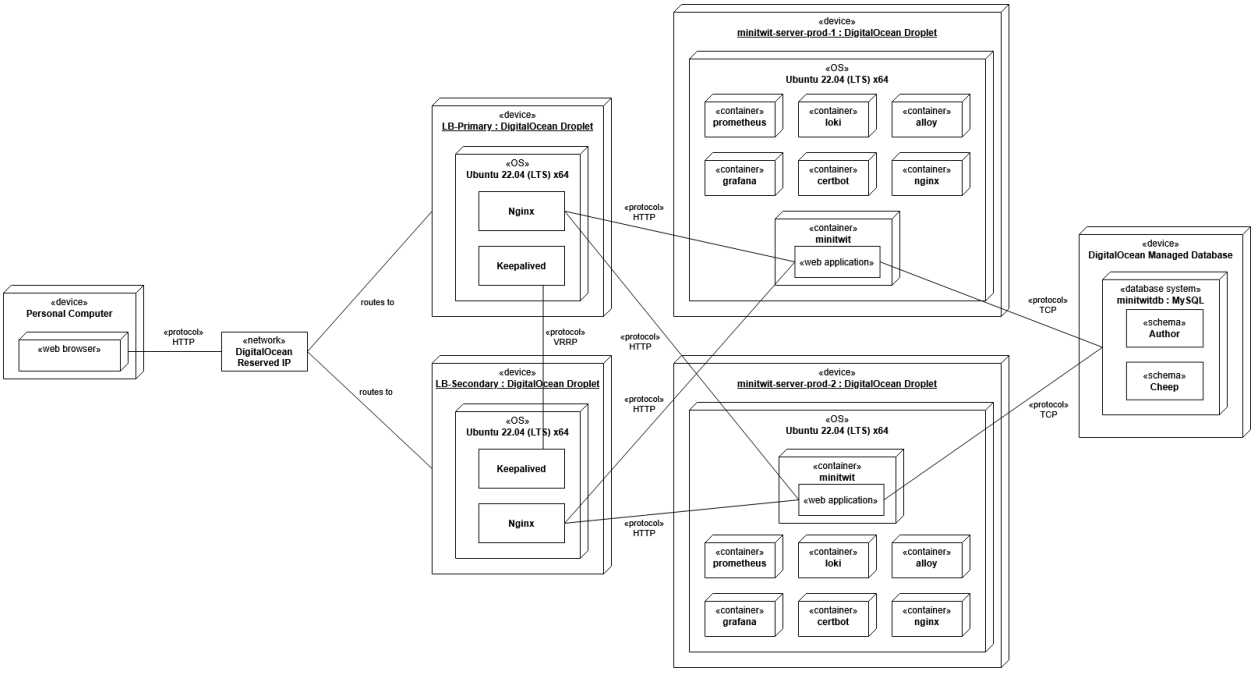
The dual load balancer (LB) setup was chosen for this reason. The primary load balancer initially handles all traffic, but is replaced by the secondary in case of failure. This is enabled through the heartbeat messages exchanged between the two using VRRP. The secondary then takes ownership of the reserved IP and resumes operation. Essentially, one LB is active while the other is on standby, which keeps the system online and reduces one potential single point of failure.

The same principle is applied to the application layer, by using two MiniTwit production servers. Both run the exact same services and both are connected to the same database, which enables traffic to be served by either instance. This provides redundancy in case one server fails, allowing the team to bring it back up while the other instance continues to handle traffic.

However, there is one major drawback to this setup. The monitoring stack is duplicated as well. Logs and metrics for each server are separate, which compromises observability consistency. If one server goes down, logs and metrics may be lost. This could be fixed by having a centralized monitoring stack e.g. on a separate device that all application nodes export to.

Conversely, since both production serves run identical containerized environments, the system is easily reproducible and scalable.

While the monitoring and logging data is not strictly centralized, the app database itself is. DigitalOcean provides a service for database hosting reducing the maintenance burden on the group. This allows focus on other aspects of the project rather than database administration, in exchange for some loss of control.



MiniTwit Deployment Diagram

1.3 Dependencies

Authors: Marie, Frederik, Nikolej, Sara

Besides the .NET packages used to make the MiniTwit application run, which can be seen here. The most important dependencies can be seen below, categorized by type.

Note: Some, like Docker, can fall into more than one category.

Workflow

- GitHub Actions - CI/CD automation platform to run workflows
- rsync - File synchronization tool

Deployed

- DigitalOcean - Cloud hosting provider used to host servers and the database
- Docker - Containerization platform
- Docker Hub - Cloud-based registry for storing and sharing Docker Images
- Nginx - High performance web server and reverse proxy
- Keepalived - High availability and failover service
- Certbot - Tool for automatically issuing and renewing SSL/TLS certificates
- Grafana - Visualization platform for logs and monitoring

- Prometheus - Monitoring service used to collect and store metrics from infrastructure and applications
- Loki - Log aggregation and storage system
- Alloy - Telemetry collector used to collect and forward logs
- Ubuntu - Linux distribution used on our servers
- MySQL - Relational database management system
- Vagrant - Tool for creating and managing reproducible virtualized development environments

Static Analysis

- Hadolint - Linter for Dockerfiles
- CSharpier - C# code formatter
- CodeQL - Static analysis engine used to identify security vulnerabilities and code quality issues
- Roslyn - .NET compiler platform for code analysis
- Docker Scout - Security and vulnerability analysis tool for Docker
- SonarCloud - Cloud-based code quality and security analysis platform
- Codacy - Automated code review and quality monitoring platform

Development

- Git - Distributed version control system
- GitHub - Web-based platform for hosting Git repositories
- .NET - Microsoft's development platform and runtime used to make the minitwit application

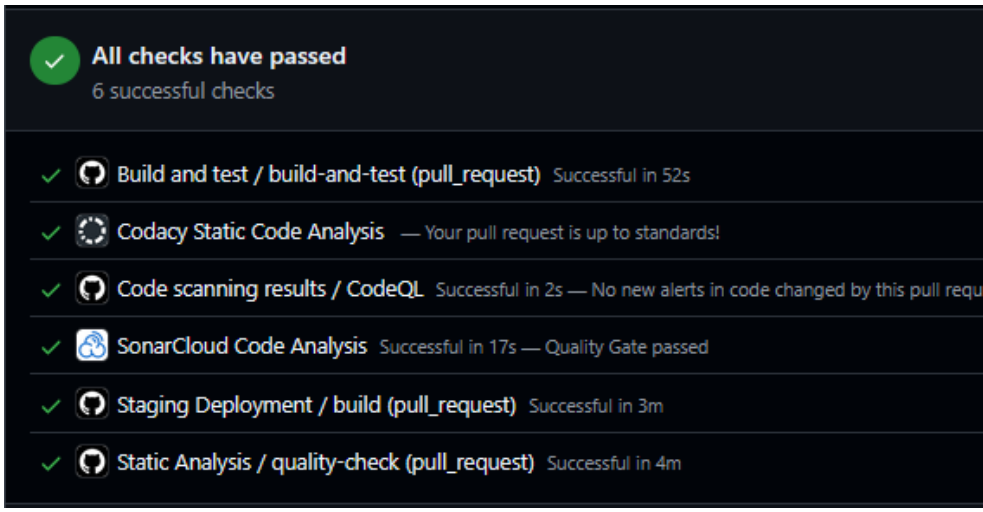
1.4 System States

Authors: Vitus, Sara

The project is currently in a strong state with respect to software quality. From the outset, we established a workflow in which all incoming changes were required to pass automated tests and static analysis checks before being merged into the production branch. Several analysis tools were also configured with strict settings to enforce high coding standards. At present, the entire project complies with these quality requirements and can thus be considered stable. Any crashes or critical issues encountered during development were thoroughly investigated, and the underlying causes were addressed accordingly.

There have been a few necessary compromises to these quality gating rules. The project is developed on top of the Chirp! platform, which was originally designed for .NET 8. Since then, newer .NET versions have been released, and .NET 8 is not considered outdated. As a result, the Docker Scout analysis consistently recommends upgrading the runtime environment. Given the project constraints and dependencies on the underlying platform, we chose to explicitly exclude this specific category from the analysis checks. While this represents a deviation from the otherwise strict quality requirements, it is not considered a critical issue as long as the limitation is acknowledged and taken into account during maintenance and future development.

Below is the latest static analysis checks.



System checks on the recent-most change

2 Process

2.1 CI/CD

Authors: Vitus, Frederik, Nikolej

The CI/CD process is based on GitHub Actions workflows that run on the `main` branch or on a scheduled timer. The MiniTwit repository currently uses the following active workflows to complete CI/CD:

- `static-analysis` — runs on push and pull request events for `main`. It performs:
  - Dockerfile linting with Hadolint against `Dockerfile-MiniTwit`.
  - .NET setup and tool restore.
  - C# formatting validation with `dotnet csharpier check .`.
  - CodeQL initialization for C#.
  - Dependency restore and a strict Roslyn build with `TreatWarningsAsErrors=true`.
  - CodeQL analysis to surface static security issues.
  - Docker Scout scanning for critical vulnerabilities on the DockerHub image.
- `build-and-test` — runs on push and pull request events for `main`. It restores dependencies, performs a clean build, and runs the test suite.
- `SonarCloud & Codacy` — provides external static analysis, code smell detection, and quality gating on Pull Requests.
- `Deploy to Staging VM` — runs on push and pull request events for `main` after the primary checks pass. Builds and pushes the Docker images and deploys them to the DigitalOcean staging Droplet.
- `Deploy To DO` — runs on pushes to `main` and can also be triggered manually. It builds and pushes the Docker images and deploys them to the DigitalOcean production Droplets.
- `automatic-weekly-release` — runs on schedule every Tuesday at 08:00 UTC and can be started manually. It builds the project, runs tests, packages release artifacts, and creates a GitHub release.

CI-CD pipeline

2.2 Monitoring

Authors: Marie, Sara, Nikolej

Monitoring is set up using Prometheus ("Prometheus Documentation" 2026) and Grafana ("Grafana Documentation" 2026) for visualizing and querying metrics, by adding the `app.MapMetrics();` and `app.UseHttpMetrics();` middleware the pipeline. `app.MapMetrics();` exposes the HTTP endpoint for Prometheus to scrape and `app.UseHttpMetrics();` collects Prometheus metrics for processed HTTP requests (from documentation of UseHttpMetrics).

The setup is pull-based as the application exposes metrics which are then pulled by Prometheus.

In Grafana, the monitoring has been split up into two dashboards; application metrics and infrastructure metrics (see video Monitoring Dashboards). Application metrics focuses mainly on request rates. It displays CPU usage in seconds, the amount of HTTP request received, and statistics on different types of requests. Infrastructure metrics focus on the server side and display dashboards containing information about: memory usage, CPU usage and process uptime.

The application's monitoring is at the reactive level (Turnbull 2014), as only a limited set of primarily operational dashboards are provided, with the main focus being on measuring availability. The monitoring does not supply data to measure user experience or data to benefit the business side.

There are many ways monitoring could be improved. For one, database monitoring would have been especially beneficial both for the operational side and to provide metrics for the business side e.g. number of users in the system.

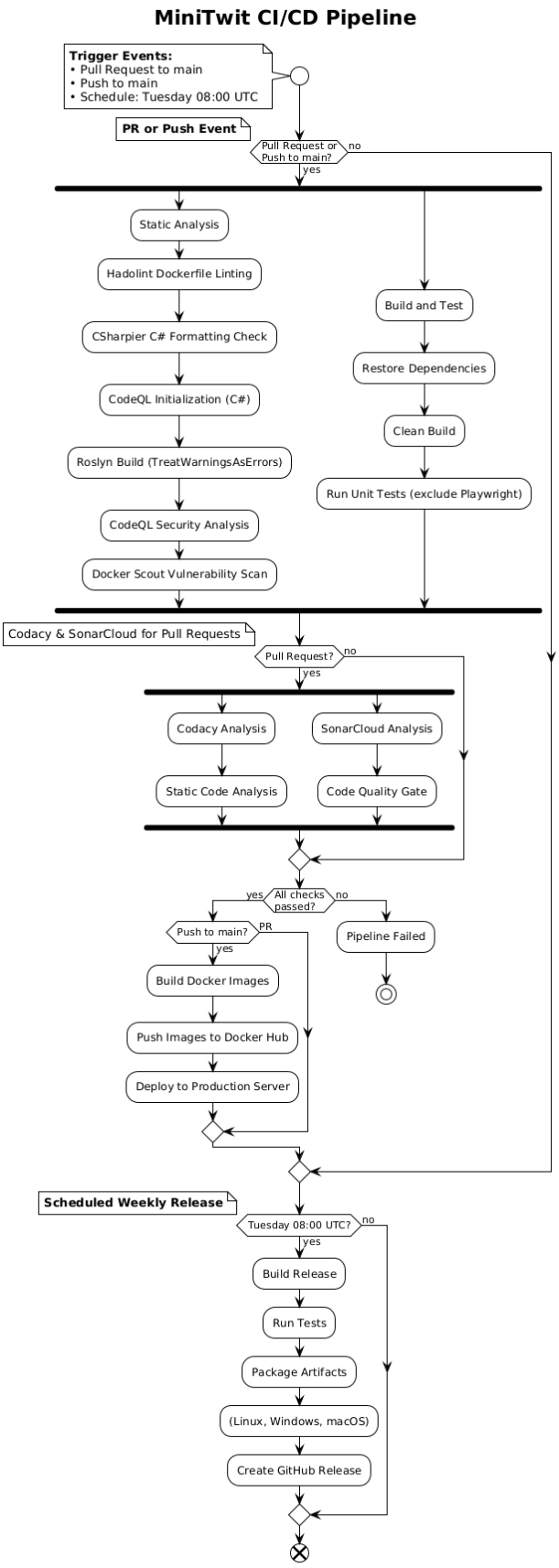


Figure 1: CI-CD pipeline

Lastly, the monitoring dashboards provided by DigitalOcean to monitor the Droplets has been regularly used.

2.3 Logging

Authors: Nikolej

The system uses a minimal push-based logging stack utilizing Alloy ("Grafana Alloy Documentation" 2026), Loki ("Grafana Loki Documentation" 2026) and Grafana ("Grafana Documentation" 2026), focusing on aggregation and visualization rather than analysis. The Grafana-Loki stack was primarily chosen because it is cheap to run and for its native support for Grafana, which the existing monitoring system was already using.

Alloy collects logs from running Docker containers through the Docker socket `host = unix:///var/run/docker.sock`, and populates them with metadata like labels before forwarding to Loki for storage. Lastly, the logs are queried by Grafana and presented visually.

Our chosen focus on aggregation and visualization is accomplished by grouping logs by containers in Grafana. The default Grafana Logs Drilldown page is more than sufficient for this purpose, hence there are no custom dashboards.

By providing a centralized view of all system logs, this setup improves observability significantly, allowing easier searching and faster response in case of errors.

Even though the logging setup was used in a limited capacity during development, it proved to be especially useful if for instance the simulation suddenly reports failed requests. The MiniTwit container logs include errors, HTTP requests and database queries, which allows us to pinpoint the problem quickly. Or if the monitoring setup indicates something unexpected, the aggregated logging platform allows us to ascertain whether it is a problem with the monitoring stack or the application itself.

2.4 Security

Authors: Sara

We applied several security hardening measures to our system across infrastructure, network, CI/CD, and container configuration.

Reverse proxy/TLS. First, we deployed a Nginx reverse proxy to the application and enabled HTTPS using TLS certificates. The reverse proxy terminates incoming HTTPS traffic and forwards requests internally to the application containers. This improves security by encrypting communication between clients and the server and by reducing direct exposure of the application itself. We used Let's Encrypt certificates together with automatic renewal through Certbot to avoid manual certificate management and ensure continued HTTPS availability.

CI/CD. In the CI/CD pipeline, we integrated automated security analysis tools to support a shift-left security approach, where vulnerabilities are detected before deployment. We added GitHub CodeQL analysis to statically scan the application source code for known security vulnerabilities and insecure coding patterns. Additionally, we integrated Docker Scout to scan container images for known common vulnerabilities and exposures and vulnerable dependencies. The pipeline was configured to fail on critical vulnerabilities, preventing insecure images from being deployed automatically.

Docker hardened images. We also hardened our Docker environment by switching several production services to Docker Hardened Images (DHI). Specifically, we replaced standard Grafana, Prometheus, and ASP.NET images with hardened variants from dhi.io. These images are designed to reduce attack surfaces by minimizing unnecessary packages and dependencies.

Container execution. Beyond changing base images, we further hardened container execution. For example, Grafana was configured to run as a non-root user instead of running with root privileges.
We also introduced initialization steps to ensure correct permissions on mounted volumes without granting unnecessary privileges to the running application containers.

A key lesson from the course was that security should not rely on a single mechanism. Therefore, our approach combined multiple layers of protection: encrypted communication through TLS, restricted network access through firewalls, automated vulnerability scanning in CI/CD, hardened container images, and safer runtime configurations. This follows a defense-in-depth strategy, where multiple independent security mechanisms reduce the likelihood that a single vulnerability compromises the entire system.

2.5 Availability and Scaling

Authors: Frederik, Nikolej, Sara, Marie

To increase the availability of the system, a simple load balancing setup was implemented (see the diagram in the deployment section). One primary load balancer initially handles all traffic, but is automatically replaced by a secondary backup in case of failure. Additionally, the system was scaled to run two application instances simultaneously on seperate DigitalOcean Droplets.

This could be improved by running at least 2 instances of the application on each of the two Droplets, and subsequently implementing an update strategy in the CI/CD pipeline. For instance, the blue-green upgrade strategy.

3 Reflection

3.1 Evolution and refactoring

Authors: Frederik, Sara, Nikolej

For the simulator to function correctly, several new endpoints specified in the Swagger documentation had to be implemented. To simplify this process, the OpenAPI Generator CLI tool was used to generate the initial endpoint structure, after which the individual endpoints were adapted to meet the required functionality and expected request behavior. Furthermore, token-based authorization was implemented for the endpoints that required authentication.

The system used a SQLite database running in the MiniTwit application. As a consequence, all persisted data was lost whenever a new deployment of the application occurred. There was therefore a need to migrate to a new database type to enable persitance. It was chosen to migrate to a MySQL database which was hosted on DigitalOcean. This was chosen due to its relative simplicity and time effectiveness.

3.2 Operations

Authors: Frederik, Marie

Monitoring dashboards and application logs were checked regularly to detect excessive system strain and identify any runtime errors. In addition, the simulator status page was monitored to identify failures or inconsistencies that were not captured by the existing logging infrastructure.

3.3 Maintenance

Authors: Frederik, Sara

During the period following the start of the simulator, several errors and faults were identified. User registration performed by the simulator was not functioning as intended, authentication for multiple endpoints behaved incorrectly, and several endpoints returned invalid response types or response bodies. To fix these issues, swarming was used to quickly discover where things were going wrong. A complete overview of the issues can be found at #29, #32, #34 and #41

We were notified that Grafana was returning an internal server error when users attempted to log in. Investigation revealed that Prometheus had exhausted the storage capacity of the droplet, preventing Grafana from accessing its internal volume correctly.

To resolve the issue, a Docker data directory used for temporary storage was cleared, after which the Docker containers on the virtual machine were restarted. To prevent similar incidents in the future, storage retention policies were configured for Prometheus, and Prometheus was assigned a dedicated volume for persistent storage. To see the whole bug report see issue #81.

After discovering that the application was unavailable, the first step was to inspect the logs. The logs revealed that the outage was caused by thread pool starvation. Further investigation showed that certain requests to the application was taking upwards of 46 minutes. By analyzing at the database activity, it became evident that queries were scanning more than 200,000 rows per second. It was therefore decided that the database required proper indexing to mitigate the issue. After the necessary indexes were added and the application was restarted, the system returned to stable operation and performed as expected. The full bug report can be found on issue #99

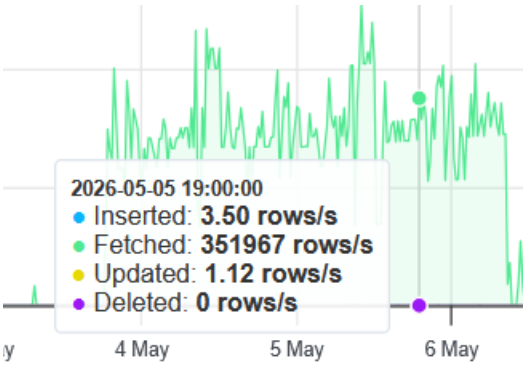


Figure 2: Database load

Database load before indexing.

3.4 Reflect and describe what was the "DevOps" style of work

Authors: Sara, Marie, Nikolej

Compared to earlier software projects, this course introduced us to several DevOps practices that changed both our workflow and our understanding of software development. Many of these practices reflected the principles behind the Three Ways of DevOps(Kim et al. 2021): improving flow, enabling fast feedback, and encouraging continuous improvement through automation, monitoring, and maintenance. A major difference compared to earlier projects was the amount of automation involved in the workflow. Tasks such as testing, linting, building containers, generating reports, and deployment were automated through pipelines and scripts. This reduced repetitive manual work and improved consistency across the project.

Continuous Integration (CI) Automatically running tests/linting on every pull request was a major change compared to earlier projects. Previously, broken code would mainly be picked up manually during pull request reviews, which depended heavily on reviewers noticing issues. Now, with CI pipelines in place, running automatic tests, linting, static analysis tools etc. caught issues early with fast automated feedback. This improved confidence when merging code and reduced the risk of introducing bugs into production.

Continuous Deployment (CD) Automated deployment significantly improved the deployment process compared to earlier projects, where the steps toward deployment were manual and inconsistent. Using CD made our deployments faster, easily reproducable and less error-prone. At the same time, setting up this deployment infrastructure was not without faults.

Monitoring & Software Maintenance Monitoring through collecting logs and metrics made a big difference for us compared to earlier projects. Using tools such as Grafana and Prometheus gave us a much better understading of the system's runtime behaviour, and once set up correctly, the logs made a big difference in debugging. Identifying bottlenecks or failures that would otherwise have been difficult to detect, became

significantly easier and made us focus more on maintaining the software in production, rather than only focussing on implementing functionality. Reliability and stability became important parts of the development process rather than something considered only at the end.

Infrastructure as Code (IaC) Luckily, we were never forced to take the whole system down and bring it back online. Towards the end of the project, we did attempt to ensure `vagrant up` could do exactly that, recreating our infrastructure from scratch. Ultimately, there were some issues that we did not have time to fix, hence the pull request #90 remains open. A video demo of `vagrant up` can be found here.

4 Use of Generative AI

Authors: Nikolej, Sara

During the development of this project we used ChatGPT in three main ways:

Debugging. When encountering unexpected bugs and unclear error messages, we often consulted ChatGPT about possible causes and fixes. While it rarely solved issues entirely on its own, it frequently helped narrow down the problem space and suggested relevant debugging strategies.

Research. This course presents challenges involving numerous technologies, many of which we were unfamiliar with, including monitoring tools, docker and load balancing. ChatGPT was used to summarize lengthy documentation, explain unfamiliar concepts, and provide concrete guides tailored to our situation.

Generating boilerplate / configurations. ChatGPT was also used for simple, repetitive tasks such as generating boilerplate code or writing Github Actions Workflow files. For example, the `build-report.yml` workflow, that converts the markdown source into a pdf.

The use of AI has made these tasks easier, spared us many frustrations during troubleshooting and saved considerable time during development. At the same time, we met certain limitations when using AI. Suggested fixes sometimes appeared plausible while being incorrect or incompatible with our setup. Additionally, relying on AI summaries may have reduced some of the deeper understanding that can come from manually reading documentation or solving problems independently, like spending hours solving a tiny bug. As a result, we found that generative AI was most useful as a supporting tool rather than a replacement for critical thinking, testing, and technical understanding.

5 References

"Grafana Alloy Documentation." 2026. 2026. https://grafana.com/docs/alloy/latest/.

"Grafana Documentation." 2026. 2026. https://grafana.com/docs/.

"Grafana Loki Documentation." 2026. 2026. https://grafana.com/docs/loki/latest/.

Kim, Gene, Jez Humble, Patrick Debois, John Willis, and Nicole Forsgren. 2021. *The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations*. It Revolution.

Planteig, C., F. H. Petersen, M. Johansen, N. Lundquist, and S. Bagger. 2025. "Chirp!" https://github.com/ITU-BDSA2025-GROUP8/Chirp.

"Prometheus Documentation." 2026. 2026. https://prometheus.io/docs/introduction/overview/.

Turnbull, James. 2014. *The Art of Monitoring*. James Turnbull.

6 Appendix

6.1 .NET dependencies

Dependencies of ITU-MiniTwit according to `dotnet list package`:

Project 'MiniTwit.Core' has the following package references [net8.0]: No packages were found for this framework.

Project 'MiniTwit.Infrastructure' has the following package references
[net8.0]: Top-level Package Requested Resolved

Microsoft.AspNetCore.Identity.EntityFrameworkCore 8.0.21 8.0.21
Microsoft.Data.Sqlite 8.0.8 8.0.8
Microsoft.Data.Sqlite.Core 8.0.8 8.0.8
Microsoft.EntityFrameworkCore 9.0.0 9.0.0
Microsoft.EntityFrameworkCore.Design 9.0.0 9.0.0
Microsoft.EntityFrameworkCore.Sqlite 8.0.8 8.0.8
Microsoft.EntityFrameworkCore.Tools 9.0.0 9.0.0
Pomelo.EntityFrameworkCore.MySql 9.0.0 9.0.0
SQLitePCLRaw.bundle_e_sqlite3 3.0.2 3.0.2

Project 'MiniTwit.Web' has the following package references [net8.0]: Top-level Package Requested Resolved

AspNet.Security.OAuth.GitHub 8.3.0 8.3.0
DotNetEnv 3.1.1 3.1.1
Microsoft.AspNetCore.Identity.EntityFrameworkCore 8.0.21 8.0.21
Microsoft.AspNetCore.Identity.UI 8.0.21 8.0.21
Microsoft.Data.Sqlite 8.0.8 8.0.8
Microsoft.Data.Sqlite.Core 8.0.8 8.0.8
Microsoft.EntityFrameworkCore 9.0.0 9.0.0
Microsoft.EntityFrameworkCore.Design 9.0.0 9.0.0
Microsoft.EntityFrameworkCore.Sqlite 8.0.8 8.0.8
Microsoft.EntityFrameworkCore.Tools 9.0.0 9.0.0
Microsoft.Playwright.NUnit 1.43.0 1.43.0
Microsoft.VisualStudio.Web.CodeGeneration.Design 9.0.0 9.0.0
Pomelo.EntityFrameworkCore.MySql 9.0.0 9.0.0
prometheus-net.AspNetCore 8.2.1 8.2.1
Serilog.AspNetCore 8.0.3 8.0.3
Serilog.Sinks.Console 6.1.1 6.1.1
Serilog.Sinks.Grafana.Loki 8.3.2 8.3.2
SQLitePCLRaw.bundle_e_sqlite3 3.0.2 3.0.2
Swashbuckle.AspNetCore 10.1.2 10.1.2
Swashbuckle.AspNetCore.Annotations 10.1.2 10.1.2
Swashbuckle.AspNetCore.SwaggerGen 10.1.2 10.1.2